OurChat.UI Project Report

- Cover Page
- Project Overview
- Project Statement
- Project Objectives
- Features
- Technology Stack
- Project Architecture
- Folder Structure
- Concepts used
- Screens (Screenshots of pages)
- Conclusion

OURCHAT.UI

A React-Based Messaging Interface Prototype with AI Assistant Integration
Project Report

Submitted By
Jiya
Mahak
Nitish

Submitted To
-

Technologies Used
- React JS
- JavaScript (ES6+)
- HTML5
- CSS3

Submission Date
-

Organization / Institute Name
-

Project Overview

OurChat.UI is a messaging interface **prototype developed using React JS.** The idea behind the project is to create a single platform where users can communicate with people and also interact with an AI Assistant within the same interface.

The project includes a **landing page** that introduces the product and a chat application where users can switch between conversations, send messages, and interact with an AI Assistant. The main goal is to simulate the experience of a modern messaging application while focusing on frontend development and user interface design.

This project is being developed as a **frontend-only prototype.** It does not include backend services, databases, user authentication, or real-time communication. Instead, the focus is on building a clean, responsive, and **interactive user interface using React.**

Through this project, we aim to **understand how modern chat applications are structured, how different UI components work together, and how React can be used to build dynamic and reusable interfaces.** The project also provides hands-on experience with component-based architecture, state management, event handling, and responsive design.

Project Statement

Modern communication is spread across different platforms for messaging, collaboration, and AI assistance. The purpose of OurChat.UI is to explore the **idea of bringing conversations and AI interactions into a single, unified interface**. This project focuses on designing and developing a modern messaging interface prototype using React JS while understanding the fundamentals of frontend application development.

Project Objectives

- To build a responsive messaging interface using React JS.
- To create a clean and user-friendly chat experience.
- To implement chat switching and message sending functionality.
- To integrate an AI Assistant chat within the same interface.
- To understand React concepts such as components, props, state, and event handling.
- To gain practical experience in building frontend applications.

Features

1. **Landing Page:** Introduces the OurChat.UI platform, its purpose, and key features.
2. **Chat List Sidebar:** Displays available conversations for quick and easy navigation.
3. **Chat Switching:** Allows users to switch between different conversations seamlessly.
4. **Message Sending:** Enables users to type and send messages within a chat.
5. **AI Assistant Chat:** Provides a dedicated chat interface for interacting with predefined AI responses.
6. **Dynamic Message Rendering:** Displays messages instantly within the conversation area.
7. **Search Chats:** Allows users to search and filter conversations efficiently.
8. **Responsive Design:** Ensures a smooth experience across desktop, tablet, and mobile devices.
9. **Modern User Interface:** Offers a clean and user-friendly design inspired by modern messaging applications.

Technology Stack

Frontend Technologies

- **React JS**
  Used to build the user interface using reusable components and manage the application's state.
- **JavaScript (ES6+)**
  Used for implementing application logic, event handling, data manipulation, and interactive functionality.
- **HTML5**
  Used to create the structure and content of the application.
- **CSS3**
  Used for styling, layout design, responsiveness, and enhancing the overall user experience.

Development Tools

- **Vite**
  Used as the development environment and build tool for creating and running the React application efficiently.
- **Git**
  Used for version control and tracking project changes during development.
- **GitHub**
  Used for project collaboration and code repository management.

Project Architecture

The OurChat.UI application follows a simple frontend architecture built using React JS. The application consists of two main sections: a **Landing Page** and a **Messaging Interface**.

Architecture Flow

User
↓
Landing Page
↓
Enter Chat Application
↓
OurChat.UI Interface
├── Sidebar
│ ├── Search Chats
│ ├── Human Chat
│ └── AI Assistant Chat
│
└── Chat Window
├── Chat Header
├── Message Area
└── Message Input

Component Structure

App
├── Landing Page
├── Sidebar
├── Chat Window
└── Message Input

Working Flow

1. The user opens the landing page.
2. The user enters the chat application.
3. The sidebar displays available conversations.
4. The user selects a chat.
5. Messages are displayed in the chat window.
6. The user types and sends a message.
7. If the AI Assistant chat is selected, predefined responses are displayed.
8. All interactions are handled on the frontend using React state.

Folder Structure

```
ourchat-ui/

├── public/
│
├── src/
│   │
│   ├── components/
│   │   ├── Sidebar.jsx
│   │   ├── ChatWindow.jsx
│   │   └── MessageInput.jsx
│   │
│   ├── data/
│   │   └── aiResponses.js
│   │
│   ├── App.jsx
│   ├── App.css
│   ├── main.jsx
│   └── index.css
│
├── package.json
├── vite.config.js
└── README.md
```

Concepts Used

HTML Concepts

- **Semantic Structure**

HTML semantic elements were used to organize the application into meaningful sections such as the landing page, sidebar, chat area, and input section.

- **Forms and Input Elements**

Input fields and buttons were used to allow users to type and send messages within the chat interface.

- **Lists and Containers**

List structures and container elements were used to display conversations and organize content on the screen.

CSS Concepts

- **Flexbox Layout**

Flexbox was used to create the overall application layout, including the sidebar, chat window, and message input section.

- **Responsive Design**

Responsive design techniques were used to ensure the application adapts properly to different screen sizes, including desktops, tablets, and mobile devices.

- **Styling and Visual Hierarchy**

CSS was used to create a clean and modern user interface through spacing, typography, colors, and alignment.

- **Hover Effects and Transitions**

Interactive effects were added to buttons and chat items to improve the user experience.

JavaScript Concepts

- **Variables and Data Structures**

Variables, arrays, and objects were used to store chat information, messages, and AI responses.

- **Functions**

Functions were used to handle actions such as sending messages, switching chats, and generating AI responses.

- **Conditional Statements**

Conditional logic was implemented to handle different application states and AI interactions.

- **Array Methods**

Methods such as map() and filter() were used to display chat data dynamically and implement search functionality.

- **Event Handling**

JavaScript event handling was used to respond to user actions such as button clicks and input changes.

React Concepts

- **Functional Components**

The application was divided into reusable components such as Sidebar, ChatWindow, and MessageInput to improve code organization and maintainability.

- **JSX**

JSX was used to combine HTML-like syntax with JavaScript, enabling dynamic rendering of user interface elements.

- **Props**

Props were used to pass data between components and maintain communication throughout the application.

- **State Management (useState)**

React's useState hook was used to manage dynamic data such as the selected chat, messages, and user input.

- **Event Handling**

React event handlers were used to manage user interactions such as selecting chats and sending messages.

- **Conditional Rendering**

Conditional rendering was used to display different content based on the selected conversation and application state.

- **List Rendering**

The map() method was used within React to dynamically render chat lists and messages from data structures.

- **Component-Based Architecture**

The application follows a component-based structure, allowing the interface to be broken down into smaller, reusable, and manageable parts.

Conclusion

The OurChat.UI project provided practical experience in building a modern messaging interface using React JS. Through this project, we explored how component-based architecture, state management, event handling, and dynamic rendering work together to create interactive user interfaces.

The application successfully demonstrates the core features of a messaging platform, including chat navigation, message sending, AI Assistant interaction, and responsive design. Although the project is a frontend-only prototype, it helped us understand the structure and workflow of real-world chat applications.

Overall, this project strengthened our understanding of React fundamentals, frontend development practices, and user interface design while providing valuable hands-on development experience.